

Mining Intelligence Catalog

Complete Technical Documentation

What this database does. How it does it. Every data point it captures. How it integrates with Mining Guardian. And how it will power the next generation of autonomous mining intelligence.

Owner: Bobby (Rob Fiesler)

Product: Mining Guardian — AI-First Bitcoin Mining Fleet Management

Database Engine: PostgreSQL 16 on ROBS-PC

Network Access: LAN: 192.168.188.47:5432 | Tailscale: 100.110.87.1:5432

Schema Version: Base + V2 + V3 (April 2026)

Total Tables: 90

Total Columns: 2363

Schemas: 10 (knowledge, hardware, firmware, ops, market, repair, pool, facility, regulatory, seed)

Design Horizon: 10 years

Fleet: 58 liquid-cooled miners, BiXBiT USA Fort Worth TX

Date: April 11, 2026

Table of Contents

1. What This Database Does
2. The Two-Database Architecture
3. How It Ties Into Mining Guardian
4. The Auto-Discovery Mechanism
5. Complete Schema Map — Every Table, Every Purpose
6. Data Point Inventory by Category
7. How Data Flows Through the System
8. What This Enables for Mining Guardian
9. Industry Context and Competitive Advantage
10. Implementation Roadmap
11. Container Monitor Field Mapping
12. knowledge.json Promotion Workflow
13. Repair Data Ingestion
14. Data Protection
15. Security
- A. SQL Query Examples

1. What This Database Does

The Mining Intelligence Catalog is a PostgreSQL 16 reference database that stores everything Mining Guardian needs to know about the world of Bitcoin mining — but does not collect in real time. It is the AI system's long-term memory, research library, and decision-making reference.

Think of it this way: Mining Guardian's operational database (`guardian.db`) records what is happening right now — this miner's hashrate is 145 TH/s, this board's temperature is 72°C, this pool rejected 3 shares. The Intelligence Catalog answers a different question: is that normal? Is it good? What should we do about it? What has happened before when we saw this pattern?

The Core Mission

The Intelligence Catalog exists to give Mining Guardian's AI the context it needs to make intelligent decisions. Without it, the AI can see data but cannot interpret it. With it, the AI knows:

- That a BM1366 chip running at 74°C in immersion cooling is perfectly healthy, but a BM1362 at the same temperature needs investigation
- That an APW12 PSU from the Q2 2023 production batch has a 5.2% failure rate on capacitor C44, so flag it for early replacement
- That Fort Worth hits 105°F in July, so expect 3-5% hashrate decline and adjust baselines accordingly
- That the BiXBiT container's coolant conductivity above 200 $\mu\text{S}/\text{cm}$ means the fluid is degrading and needs replacement
- That ERCOT 4CP events happen June-September between 2-7pm, and curtailing during those 15-minute windows saves tens of thousands in demand charges
- That a hashboard with signal chain RX absent at chip position 14 means a dead ASIC at that domain boundary
- That this specific S19J Pro's hashrate has been 2 standard deviations below baseline for 72 hours — predict failure within 5 days

None of this knowledge exists in the operational database. It lives here.

Design Philosophy

"Capture everything. Discard nothing." — Bobby's founding principle. This database is designed on a 10-year horizon. Every row tracks its data source. Every text field is full-text searchable. Every model name is fuzzy-matchable. Every data point carries a confidence level and citation chain back to its original source.

The second principle is equally critical: auto-discovery. If Mining Guardian encounters a data point it has never seen before — a new API field from a firmware update, a new metric from a container sensor, an unknown error code — the system must never skip it. It must register it, log it, attempt to classify it, and flag it for human review. No data point is too small or insignificant.

2. The Two-Database Architecture

Mining Guardian operates on two databases with fundamentally different purposes:

Attribute	guardian.db (Operational)	Intelligence Catalog (Reference)
Engine	SQLite 3	PostgreSQL 16
Location	VPS / Mac mini (production)	ROBS-PC (192.168.188.47)
Purpose	Real-time operational telemetry	Reference intelligence, AI context
Data freshness	Updated every 5-minute scan	Updated as knowledge is discovered
Size	17 tables, ~10M+ rows	90 tables, growing reference data
Access pattern	High-frequency writes, time-series	Read-heavy, JOIN-intensive lookups
Who writes	Mining Guardian scan loop	Research imports, LLM discoveries, Bobby
Who reads	Dashboard, Grafana, LLM, Slack bot	LLM context builder, decision engine
Retention	Rolling (prune old scans)	Permanent (10-year horizon)

guardian.db Operational Tables

These 17 tables capture everything happening in real time:

Table	Purpose	Key Data
scans	Scan metadata	Scan ID, timestamp, duration
miner_readings	Per-miner per-scan telemetry	Hashrate, temp, power, firmware, status, errors
chain_readings	Per-hashboard per-scan	Board hashrate, voltage, frequency, HW errors, temps
chip_readings	Per-chip per-scan	Chip frequency, voltage, temperature
pool_readings	Per-pool per-miner per-scan	Accepted/rejected shares, difficulty, pool URL
miner_state_readings	Extended miner state	Medium/low hashrate, max consumption, cooling mode
miner_hardware	Hardware identity (permanent)	Board serial, chip die, PCB version, BOM, control board
miner_ams_extended	AMS extended fields	Map location, PDU counter, stratum URL
hvac_readings	HVAC system telemetry	Supply/return temp, delta-T, VFD %, faults, alarms
weather_readings	Ambient conditions	Temperature, humidity, feels-like, highs/lows
log_metrics	Parsed miner log data	PSU voltage, chain events, system health
miner_logs	Raw log collection	Full miner.log content per collection
miner_restarts	Restart history + outcomes	Restart type, hashrate before/after, recovery time

Table	Purpose	Key Data
known_dead_boards	Suppressed dead boards	Board index, ticket status, suppression
pending_approvals	Actions awaiting Bobby	Miner, action type, Slack thread
action_audit_log	Complete action history	Action taken, decision, approved by, notes
ams_notifications	AMS alert tracking	Alert type, miner, timestamp

Additional Operational Data Sources

Beyond guardian.db, Mining Guardian collects data from two additional systems that feed into the intelligence catalog:

- Container Monitor (container_monitor.py): Polls the BiXBiT container management system every scan cycle. Collects 30+ data points across 5 categories — hydraulics (supply/return temp and pressure, flow rate, conductivity, filter differentials), cooling equipment (dry cooler frequency, fan status, pump frequencies), power (PMM1/PMM2/PMM3 kW, ASIC total, infrastructure, PUE), environment (inside temps, cabinet temps, outside conditions), and safety (tank level, leakage, smoke, alarms).
- HVAC Client (hvac_client.py): Communicates with the facility HVAC system via BACnet protocol. Collects supply/return temperatures, differential pressure, spray pump status, cooling tower fan VFD percentages, chilled water pump VFD percentages, fault statuses, vibration alarms, and basin levels.
- knowledge.json: Persistent LLM memory file. Stores fleet summary, miner profiles, known issues, discovered patterns, prediction accuracy, fingerprints, daily deep dive results, and operator rules. Updated every scan cycle by the knowledge manager.

3. How It Ties Into Mining Guardian

Mining Guardian runs 8 AI features in its scan loop, plus weekly deep training. The Intelligence Catalog provides the reference intelligence that makes each feature smarter.

Feature 1: Outcome Feedback Loop

After every restart, the system checks whether the miner recovered. The Intelligence Catalog provides baseline hashrate expectations per model/firmware/cooling combo (`ops.miner_baseline_reference`) so the system knows what 'recovered' actually means. If a restarted S19J Pro comes back at 110 TH/s, is that success? The catalog says nominal for that config under BiXBiT firmware in immersion is 145 TH/s — so no, that is a partial recovery.

Feature 5: HVAC/Environment Correlation

The HVAC correlator links facility conditions to miner performance. The Intelligence Catalog provides reference ranges (`facility.hvac_patterns`, `facility.container_hydraulics_reference`) so the AI knows when supply temperature, delta-T, or differential pressure is outside normal bounds. It also provides historical weather baselines (`facility.weather_reference`) and quantified performance impact curves (`ops.environmental_correlations`) so the AI can separate 'it is hot outside' from 'something is wrong with the cooling system.'

Feature 6: Pre-Failure Prediction

The predictor analyzes 12+ signals per miner to predict failures 24-72 hours in advance. The Intelligence Catalog feeds this with: failure pattern intelligence (`ops.failure_patterns`, `ops.failure_symptoms`), chip-level diagnostic reference data (`hardware.chips` diode-mode and domain resistance), hashboard signal chain patterns (`hardware.signal_chain_reference`), known defects per PCB revision (`hardware.hashboards`), and failure rates by production batch (`hardware.psu_serial_batches`, `hardware.board_serial_batches`).

Feature 7: Fingerprint Builder

Builds a permanent identity profile for each miner combining hardware serial numbers, chip bins, PCB versions, performance baselines, and behavioral patterns. The Intelligence Catalog provides chip bin performance reference data (`hardware.chip_bins`), board serial batch quality intelligence, and model-level baseline specs so fingerprints can express deviation from expected behavior.

Feature 8: Action Diversity

Ensures the AI does not recommend the same action repeatedly. The Intelligence Catalog provides the full repair procedure library (`repair.repair_procedures`, `repair.repair_steps`) with expected success rates, required tools, and alternative approaches. If 'restart' has been tried 3 times, the catalog can suggest 'check PSU voltage rails' or 'inspect board signal chain at position N.'

Daily Deep Dive

Every day, the local LLM generates a comprehensive fleet analysis. The Intelligence Catalog provides model specs, firmware known bugs, operational thresholds, and HVAC reference data — giving the LLM the expert context it needs to produce actionable insights rather than generic observations.

Weekly Claude Training

Every Sunday at 3am, Claude API synthesizes all accumulated data into fleet-wide patterns. The Intelligence Catalog provides the structured reference framework that Claude uses to validate or challenge patterns: 'Is this failure rate normal for BM1366 chips?' (hardware.chips), 'Is this PSU batch known for issues?' (hardware.psu_serial_batches), 'Does this error code indicate firmware bug or hardware fault?' (ops.miner_error_codes, firmware.firmware_bugs).

Knowledge Manager

Maintains knowledge.json, the LLM's working memory. The Intelligence Catalog provides the validated, curated version of everything in knowledge.json — when a pattern graduates from 'observed' to 'confirmed,' it moves from knowledge.json into the Intelligence Catalog with full source citations, confidence levels, and verification status.

Overnight Automation

Between 10pm and 6am, Mining Guardian autonomously executes low-risk actions (restarts for underperforming miners). The Intelligence Catalog provides risk classification intelligence: known failure patterns that are safe to auto-restart vs. patterns that require human review (ops.failure_patterns.severity, ops.operational_thresholds). It also provides the 84°C temperature threshold and the 20-minute post-restart grace period as validated operational rules.

4. The Auto-Discovery Mechanism

Bobby's hard requirement: "If a new data point comes up that it has never seen before, it knows to mark it down, register it as a new data point, not skip over it."

This is implemented through four interconnected tables:

Step 1: The Field Registry

knowledge.field_registry is the master dictionary of every known data field across all sources. It ships pre-seeded with 75 entries covering every field from guardian.db, container_monitor, and HVAC. Each entry records:

- Field key (canonical name like 'miner_readings.hashrate')
- Source system (guardian_db, container_monitor, hvac, ams_api, miner_log)
- Category (thermal, electrical, performance, environmental, safety, identity, pool, firmware)
- Data type (numeric, text, boolean, json, timestamp)
- Unit (celsius, watts, th/s, mhz, mv, psi, mpa, %)
- Expected range (typical_min, typical_max)
- Alert thresholds (alert_threshold_low, alert_threshold_high)
- Mapping to Intelligence Catalog table and column
- Mapping to guardian.db table and column
- First/last seen timestamps and occurrence count

Step 2: Unknown Field Detection

When Mining Guardian processes any API response, log parse, or sensor reading, it extracts all field names and checks each against the field_registry. If a field is NOT found:

1. A new row is inserted into knowledge.unknown_fields with the raw field name, first observed value, detected data type, source system, endpoint, miner ID, firmware version, and parent JSON path.
2. Up to N sample values are collected in a JSONB array for pattern analysis.
3. The Tier 1 LLM (Qwen 2.5 32B) auto-classifies the field: suggests a category, unit, and confidence score with reasoning.
4. The field enters a status workflow: new → under_review → classified → mapped (or ignored/duplicate).
5. Bobby reviews weekly, or the Tier 2 Sunday analysis handles bulk classification.

Step 3: Raw Ingestion Log

knowledge.raw_ingestion_log stores the complete raw payload of every API response and log parse. This table is partitioned by quarter for performance and has a retention tier system: standard (90 days auto-prune), flagged (1 year), permanent (never delete). When something weird happens, Bobby can look at exactly what the system received — full forensic capability.

Step 4: Discovery Audit Trail

knowledge.field_discovery_log tracks the complete lifecycle of every discovered field: when it was first seen, who/what classified it, when it was mapped to a catalog table, and any schema alterations made to accommodate it. This answers the question: 'When did we first see this field, who classified it, and where does it live now?'

5. Complete Schema Map

The Intelligence Catalog contains 90 tables organized into 10 schemas with 2363 total columns. Here is every table, its purpose, and its column count.

knowledge — 10 tables, 173 columns

Foundation layer — source tracking, citations, data conflicts, freshness monitoring, auto-discovery mechanism, LLM pattern accuracy

Table	Cols	Version
knowledge.citations	14	Base
knowledge.contributors	15	Base
knowledge.data_conflicts	18	Base
knowledge.field_discovery_log	7	V3 New
knowledge.field_registry	25	V3 New
knowledge.freshness_log	12	Base
knowledge.llm_analysis_patterns	19	V3 New
knowledge.raw_ingestion_log	14	V3 New
knowledge.sources	20	Base
knowledge.unknown_fields	29	V3 New

hardware — 19 tables, 637 columns

Miner hardware intelligence — manufacturers, chips, PSUs, control boards, hashboards, miner models, aliases, spec history, compatibility, serial batches, chip bins, fan specs, pinouts, signal chains

Table	Cols	Version
hardware.board_serial_batches	20	V2 New
hardware.chip_bins	27	V2 New
hardware.chips	50	Base
hardware.connector_pinouts	15	V2 New
hardware.control_board_serial_batches	18	V2 New
hardware.control_boards	53	Base
hardware.cooling_compatibility	17	Base
hardware.fan_specifications	36	V2 New

Table	Cols	Version
hardware.hashboards	71	Base
hardware.manufacturers	21	Base
hardware.miner_models	121	Base
hardware.model_aliases	10	Base
hardware.model_known_issues	31	V2 New
hardware.model_spec_history	17	Base
hardware.psu_compatibility	11	Base
hardware.psu_models	57	Base
hardware.psu_serial_batches	18	V2 New
hardware.psu_voltage_rails	23	V2 New
hardware.signal_chain_reference	21	V3 New

firmware — 7 tables, 169 columns

Firmware intelligence — releases, compatibility matrices, API capabilities, telemetry field maps, known bugs, changelogs, autotuning profiles

Table	Cols	Version
firmware.firmware_api_capabilities	21	Base
firmware.firmware_autotuning_profiles	24	Base
firmware.firmware_bugs	27	Base
firmware.firmware_changelog	13	Base
firmware.firmware_compatibility	22	Base
firmware.firmware_releases	35	Base
firmware.firmware_telemetry_fields	27	Base

ops — 9 tables, 240 columns

Operational intelligence — failure patterns, symptoms, probabilistic diagnosis, thresholds, environmental correlations, operational profiles, alert rules, error codes, baselines

Table	Cols	Version
ops.alert_rules	25	Base
ops.environmental_correlations	35	Base
ops.failure_patterns	27	Base

Table	Cols	Version
ops.failure_symptoms	15	Base
ops.miner_baseline_reference	26	V3 New
ops.miner_error_codes	18	Base
ops.operational_profiles	31	Base
ops.operational_thresholds	51	Base
ops.symptom_pattern_map	12	Base

market — 10 tables, 239 columns

Market intelligence — user reviews, pricing history, manufacturer reputation, forum posts, teardowns, war stories, availability signals, depreciation schedules, resale values

Table	Cols	Version
market.depreciation_schedules	30	V3 New
market.forum_posts	27	Base
market.manufacturer_reputation	14	Base
market.market_availability	17	Base
market.pricing_history	26	Base
market.resale_value_history	22	V3 New
market.review_summaries	22	V2 New
market.teardown_reports	22	Base
market.user_reviews	34	Base
market.war_stories	25	Base

repair — 10 tables, 257 columns

Repair intelligence — parts catalog, suppliers, availability, repair procedures, step-by-step guides, shop directory, repair records, statistics, shop reviews, diagnostic tools

Table	Cols	Version
repair.diagnostic_tools	24	V3 New
repair.part_availability	15	Base
repair.part_suppliers	16	Base
repair.parts	28	Base
repair.repair_procedures	30	Base

Table	Cols	Version
repair.repair_records	43	Base
repair.repair_shops	35	Base
repair.repair_statistics	30	Base
repair.repair_steps	17	Base
repair.shop_reviews	19	Base

pool — 7 tables, 152 columns

Pool intelligence — pool directory, endpoints, stratum configs, reliability history, incidents, Bitcoin network snapshots, stratum error codes

Table	Cols	Version
pool.bitcoin_network_snapshots	22	Base
pool.mining_pools	43	Base
pool.pool_endpoints	17	Base
pool.pool_incidents	15	Base
pool.pool_reliability_history	19	Base
pool.stratum_configurations	24	Base
pool.stratum_error_codes	12	Base

facility — 13 tables, 395 columns

Facility infrastructure — cooling solutions, PDUs, facilities, HVAC patterns, rack positions, container hydraulics/cooling/environment reference, immersion fluids, electricity rates, demand response, curtailment, weather reference

Table	Cols	Version
facility.container_cooling_equipment	35	V3 New
facility.container_environment_reference	20	V3 New
facility.container_hydraulics_reference	37	V3 New
facility.cooling_solutions	40	Base
facility.curtailment_events	24	V3 New
facility.demand_response_programs	30	V3 New
facility.electricity_rates	31	V3 New
facility.facilities	25	Base

Table	Cols	Version
facility.hvac_patterns	47	Base
facility.immersion_fluids	44	V3 New
facility.power_distribution_units	22	Base
facility.rack_positions	15	Base
facility.weather_reference	25	V3 New

regulatory — 5 tables, 101 columns

Regulatory intelligence — legal frameworks, environmental regulations, import/export rules, insurance requirements, tax treatment and depreciation

Table	Cols	Version
regulatory.environmental_regs	20	Base
regulatory.frameworks	22	Base
regulatory.import_export_rules	26	Base
regulatory.insurance_requirements	12	Base
regulatory.tax_treatment	21	Base

6. Data Point Inventory by Category

This section catalogs every major data dimension the Intelligence Catalog captures, organized by what Bobby cares about.

Thermal Intelligence

Data Point	Where It Lives
Chip temperature reference per model	hardware.chips — tj_max_celsius, thermal_signature_profile
Board temperature baselines	ops.miner_baseline_reference — board_temp_c_baseline
Container supply/return temp norms	facility.container_hydraulics_reference — supply_temp_c_nom/min/max, return_temp_c_nom/min/max
HVAC supply/return temp reference	facility.hvac_patterns — supply_temp_f_nom/min/max, return_temp_f_nom/min/max
Delta-T reference (supply→return)	facility.container_hydraulics_reference — delta_t_c_nom/min/max
Cabinet temperature limits	facility.container_environment_reference — dist_cabinet_temp_c_max, ctrl_cabinet_temp_c_max
Weather impact on thermal performance	ops.environmental_correlations — ambient_temp_impact_th_per_c, performance_at_35c/40c/45c_pct
Cooling fluid thermal properties	facility.immersion_fluids — specific_heat_j_kg_k, thermal_conductivity_w_mk, operating_temp_range

Electrical Intelligence

Data Point	Where It Lives
PSU voltage rails (12V nom/min/max/ripple)	hardware.psu_models — output_voltage_12v_nom/min/max, ripple_mv
PSU output current (nom/max)	hardware.psu_models — output_current_a_nom/max
Board voltage reference per model	hardware.hashboards — board_voltage_v_nom/min/max
Chip voltage range and step size	hardware.chips — core_voltage_mv_min/max/nom, voltage_step_mv
Chip frequency range and step size	hardware.chips — frequency_mhz_min/max/nom, frequency_step_mhz
Chip diode-mode readings (diagnostic)	hardware.chips — diode_mode_reading_v_nom/min/max
Domain resistance reference	hardware.chips — domain_resistance_ohm_nom/min/max
Container power monitoring	facility.container_cooling_equipment — PMM layout, total_power_capacity_kw
PUE reference (target/typical/worst)	facility.container_cooling_equipment — pue_target/typical/worst_case
Electricity rate structures	facility.electricity_rates — energy/demand/transmission/distribution/ancillary charges, TOU tiers

Performance Intelligence

Data Point	Where It Lives
Hashrate baselines per model/firmware/cooling	ops.miner_baseline_reference — hashrate_th_baseline, stddev, warning/critical thresholds
Power baselines	ops.miner_baseline_reference — power_w_baseline, stddev
Efficiency baselines (J/TH)	ops.miner_baseline_reference — efficiency_j_th_baseline
Chip performance bins	hardware.chip_bins — hashrate/power/efficiency per bin, optimal voltage/frequency
Firmware autotuning profiles	firmware.firmware_autotuning_profiles — target vs measured hashrate/power/efficiency per mode
Model spec history (versioned)	hardware.model_spec_history — hashrate/power/efficiency changes over product lifetime
Operational profiles	ops.operational_profiles — complete configs per mode with profitability estimates
HW error rate baselines	ops.miner_baseline_reference — hw_error_rate_baseline

Hardware Identity Intelligence

Data Point	Where It Lives
PSU serial number formats and batches	hardware.psu_models — serial_number_format; hardware.psu_serial_batches — batch quality tracking
Board serial numbers and batches	hardware.board_serial_batches — production date, facility, failure rates, quality ratings
Control board serial batches	hardware.control_board_serial_batches — same quality tracking for control boards
Chip bins and passports	hardware.chip_bins — 27 columns of per-chip-bin performance and quality data
Connector pinouts	hardware.connector_pinouts — pin-by-pin mapping for every connector type per model
PCB revision tracking	hardware.hashboards — pcb_revision, known_defects per revision
Miner model tear sheet data	hardware.miner_models — dimensions, weight, warranty, noise, connector types, chip count, years in production

Failure and Repair Intelligence

Data Point	Where It Lives
Failure patterns (root cause KB)	ops.failure_patterns — 27 columns including category, severity, root cause, occurrence rate, repair success rate

Data Point	Where It Lives
Failure symptoms (observable indicators)	ops.failure_symptoms — detection method, API field, threshold, severity, urgency
Symptom → pattern probabilistic mapping	ops.symptom_pattern_map — probability, pathognomonic flag, required symptoms
Repair procedures (step-by-step)	repair.repair_procedures — 30 columns with tool requirements, success criteria, skill level
Parts catalog with diagnostic reference	repair.parts — test method, expected/failure readings, common failure modes, failure rates
Repair shop directory	repair.repair_shops — 35 columns with capabilities, turnaround, pricing, reputation
Repair records (1M+ capacity)	repair.repair_records — 43 columns with raw_shop_data JSONB for flexible ingestion
Diagnostic tool inventory	repair.diagnostic_tools — test fixtures, multimeters, oscilloscopes, Bobby's inventory and ratings
Signal chain reference	hardware.signal_chain_reference — per-signal expected values, test points, failure interpretation

Financial Intelligence

Data Point	Where It Lives
Pricing history (new/used, BTC-denominated)	market.pricing_history — 26 columns tracking price over time with vendor/condition/warranty data
Depreciation schedules	market.depreciation_schedules — MACRS 5yr, Section 179, bonus depreciation, pre-computed yearly amounts
Resale value tracking	market.resale_value_history — time-series by condition, with BTC price context and monthly depreciation rate
Electricity costs	facility.electricity_rates — complete rate structures with TOU tiers
Demand response revenue	facility.demand_response_programs — capacity/energy payments, estimated annual revenue
Curtailment net impact	facility.curtailment_events — payment received vs mining lost, net financial impact per event
Operational profitability profiles	ops.operational_profiles — estimated daily BTC/USD/cost/profit per mode

7. How Data Flows Through the System

Data enters Mining Guardian from multiple sources and flows through a defined pipeline before reaching the Intelligence Catalog.

Inbound Data Sources

Source	Frequency	Data Collected	Stored In
AMS/BiXBiT API	Every 5 minutes	Miner telemetry, board data, chip data, pool data, state data	guardian.db: miner_readings, chain_readings, chip_readings, pool_readings, miner_state_readings, miner_ams_extended
Miner Logs	Daily collection + on-restart	PSU voltage, chain events, system health, hardware identity	guardian.db: log_metrics, miner_hardware, miner_logs
Container Monitor	Every scan cycle	Hydraulics, cooling, power, environment, safety	Logged to Slack, correlated in AI analysis
HVAC BACnet	Every scan cycle	Supply/return temps, VFD %, faults, alarms	guardian.db: hvac_readings
Weather API	Every scan cycle	Temperature, humidity, feels-like	guardian.db: weather_readings
Slack Interactions	Event-driven	Approvals, denials, operator commands	guardian.db: action_audit_log, pending_approvals
Qwen 2.5 32B (Tier 1)	Every scan (~4.6s)	Scan analysis, insights, predictions	guardian.db: llm_analysis; knowledge.json
Claude Sonnet (Tier 2)	Weekly Sunday 3am	Pattern synthesis, knowledge merges, deep analysis	knowledge.json

Intelligence Catalog Population

The Intelligence Catalog is populated through five channels:

Research Import: Bobby or automated scripts import manufacturer specs, teardown reports, repair data, firmware release notes, pool statistics, and regulatory information. Each import is tagged with its source and confidence level via knowledge.sources and knowledge.citations.

LLM Discovery: When the Tier 1 or Tier 2 LLM identifies a new pattern, failure mode, or correlation, it can be promoted from knowledge.json into the Intelligence Catalog as a verified entry.

Auto-Discovery: When unknown fields are detected (Section 4), classified, and mapped, they expand the catalog schema. This is the only mechanism that can add entirely new data points without human intervention.

Community/Market Ingestion: Forum posts, user reviews, pricing data, and repair records from external sources flow into the market and repair schemas.

Bobby's Direct Input: Bobby can manually add or verify any data point. Entries verified by Bobby are marked `verified_by_bobby = TRUE` and carry higher confidence.

Query Flow: How the AI Uses the Catalog

When the LLM needs to make a decision, the context builder queries the Intelligence Catalog to assemble relevant reference data. Example flow for a miner showing declining hashrate:

1. Look up the miner model in `hardware.miner_models` → get stock specs and thresholds
2. Look up the firmware in `firmware.firmware_releases` → check for known bugs (`firmware.firmware_bugs`)
3. Look up baselines in `ops.miner_baseline_reference` → what should this config produce?
4. Look up failure patterns in `ops.failure_patterns` → what causes gradual hashrate decline?
5. Check environmental correlations in `ops.environmental_correlations` → is weather a factor?
6. Check PSU serial batch in `hardware.psu_serial_batches` → is this batch known for issues?
7. Look up repair procedures in `repair.repair_procedures` → what are the diagnostic steps?
8. Assemble all context into the LLM prompt → get an informed, specific diagnosis

8. What This Enables for Mining Guardian

The Intelligence Catalog transforms Mining Guardian from a monitoring tool into an autonomous mining intelligence platform. Here is what becomes possible:

Predictive Maintenance with Context

Current: the system can detect anomalies. With the catalog: the system can predict failures 24-72 hours in advance because it knows that BM1366 chips from bin designation 'B' have a 12% higher failure rate after 8,000 hours, and this miner's chips hit that mark last week. The prediction is not just 'something might fail' — it is 'chip at position 14 on board 2 is likely to fail based on domain resistance trending high relative to the catalog reference value.'

Repair-vs-Replace Decision Engine

The system can calculate: this S19J Pro's repair cost is \$150 (repair.repair_procedures → parts cost + labor estimate), its book value is \$0 (market.depreciation_schedules — fully depreciated under MACRS 5-year), its market value is \$400 (market.resale_value_history), and a replacement unit costs \$800 (market.pricing_history). Recommendation: repair. If repair fails, sell for parts and buy replacement.

Automated Curtailment Decisions

When ERCOT signals a 4CP event (facility.demand_response_programs), the system knows: curtailing 58 miners for 15 minutes loses \$X in mining revenue (calculated from ops.operational_profiles daily estimates) but saves \$Y in annual demand charges (facility.electricity_rates demand component × 12 months). If $Y > X$, curtail automatically.

Fleet-Wide Knowledge Transfer

When Mining Guardian deploys to a second customer site, the Intelligence Catalog goes with it. The new site immediately benefits from all accumulated knowledge: failure patterns, repair procedures, firmware compatibility, chip bin data. The auto-discovery mechanism learns new site-specific data points and feeds them back into the shared catalog.

Firmware Regression Detection

When miners show identical fault patterns within hours of a firmware update, the system cross-references firmware.firmware_bugs, firmware.firmware_changelog, and firmware.firmware_compatibility to determine whether this is a known regression. Operator rule: prefer 'firmware regression' diagnosis over individual hardware failure when N+ miners are affected simultaneously.

Intelligent Procurement

Bobby can query: 'Which PSU batches have the lowest failure rates?' (hardware.psu_serial_batches), 'Which immersion fluid has the best thermal properties for our operating range?' (facility.immersion_fluids), 'What is the total cost of ownership for an S21 vs. S19k Pro over 3 years?' (market.depreciation_schedules + facility.electricity_rates + market.pricing_history + repair.repair_statistics).

Continuous Self-Improvement

The LLM analysis pattern tracking (`knowledge.llm_analysis_patterns`) records prediction accuracy over time. The system learns which patterns are reliable (accuracy > 80%) and which generate false positives (accuracy < 40%). Low-accuracy patterns are automatically deprioritized. High-accuracy patterns get promoted to auto-action eligibility.

Regulatory and Tax Compliance

The regulatory schema ensures Bobby never misses a compliance requirement. When tax season arrives, `market.depreciation_schedules` has pre-computed MACRS depreciation amounts per asset. When a new tariff is announced on Chinese ASIC imports, `regulatory.import_export_rules` captures it with effective dates and duty rates.

9. Industry Context and Competitive Advantage

The Bitcoin mining fleet management market is evolving rapidly. As of April 2026, the major fleet management platforms — Luxor Commander, Foreman, Braiins Manager, ItsMiner, OptiFleet by Foundry, and HiveOS — provide monitoring, bulk configuration, and basic automation.¹ Luxor's "Intelligent Miner" feature adjusts power settings every 5 minutes based on hashrate pricing and electricity costs, claiming 8-14% profitability improvement over binary curtailment.²

What none of these platforms have is what the Intelligence Catalog provides: a deep, structured, citation-tracked knowledge base that gives the AI system genuine understanding rather than just pattern matching. The distinction matters:

Capability	Industry Standard	Mining Guardian + Intelligence Catalog
Monitoring	Real-time dashboards with hashrate, temp, power	Same, plus deep context for every reading
Anomaly detection	Static thresholds or simple ML	Model-specific, firmware-specific, cooling-specific thresholds with per-chip baselines
Failure prediction	Basic (if any)	12-signal predictor with failure pattern knowledge base, chip diagnostic reference, batch quality data
Repair guidance	None (external)	Full repair procedure library with parts, tools, step-by-step, success rates
Curtailment	Binary on/off or simple power adjustment	Multi-factor: grid price, demand charges, mining revenue, opportunity cost, per-event financial tracking
Knowledge retention	None — operator knowledge stays in heads	Permanent, citation-tracked, searchable knowledge base with auto-discovery
Unknown data handling	Ignored	Auto-discovered, classified, and integrated

The Intelligence Catalog is not just a database — it is the institutional memory of a mining operation. When Bobby hires a new technician, they do not need years of experience to diagnose a failing hashboard. They query the catalog. When Mining Guardian deploys to a new customer, the customer does not start from zero. They inherit the entire accumulated intelligence of every deployment that came before.

1. Hashrate Index, "Top 5 Bitcoin Mining Fleet Management Software for 2026," <https://hashrateindex.com/blog/top-bitcoin-mining-fleet-management-software-for-2026/>

2. Bitcoin Magazine, "Luxor Launches Commander Fleet Management Software," April 2026, <https://bitcoinmagazine.com/news/luxor-launches-commander-fleet-software>

10. Implementation Roadmap

The Intelligence Catalog is implemented in three SQL files, designed to be run in sequence on PostgreSQL 16:

File	Lines	Content	Status
intelligence_catalog_schema.sql	4,431	Base schema: 63 tables, 10 schemas, 16 enums, extensions, triggers	Complete
intelligence_catalog_schema_v2_additions.sql	887	V2: 9 new tables, 113 new columns — PSU serials, chip bins, board serials, fan specs, pinouts, known issues, reviews	Complete
intelligence_catalog_schema_v3_additions.sql	1,256	V3: 14+ new tables, 170+ columns — auto-discovery, container reference, immersion fluids, electricity, curtailment, depreciation, diagnostics, weather, model completeness	Complete

Deployment Timeline

Now: All three SQL files are ready. Schema can be deployed to PostgreSQL 16 on ROBS-PC.

May 5-9, 2026: Mac mini arrives. Mining Guardian migrates from VPS to docker-compose on Mac mini. Intelligence Catalog remains on ROBS-PC, accessible over Tailscale.

May-June 2026: Begin populating the catalog: manufacturer specs, firmware release data, failure pattern library, chip bin data from Bobby's fleet data. Auto-discovery mechanism goes live.

July 2026: NAS migration (UGREEN NASync iDX6011 Pro). Intelligence Catalog can optionally move to NAS or remain on ROBS-PC.

Ongoing: Auto-discovery continuously expands the catalog. Weekly LLM training promotes patterns from knowledge.json into the catalog. Community and market data imports grow the reference library.

Summary

Metric	Value
Total tables	90
Total columns (data points)	2363
Schemas	10
Auto-discovery tables	4
Pre-seeded field registry entries	75

Metric	Value
SQL files total lines	6,574
Design horizon	10 years
Database engine	PostgreSQL 16
Philosophy	Capture everything. Discard nothing.

11. Container Monitor Field Mapping

container_monitor.py polls the BiXBiT container management system every scan cycle and collects 30+ data points across 5 categories. Here is the explicit mapping from every container monitor field to its corresponding Intelligence Catalog table and column.

Hydraulics (10 fields)

Container Monitor Field	Sensor	Catalog Table	Catalog Column(s)
supply_temp_c	TT01	facility.container_hydraulics_reference	supply_temp_c_nom/min/max
return_temp_c	TT02	facility.container_hydraulics_reference	return_temp_c_nom/min/max
delta_t_c (computed)	TT02-TT01	facility.container_hydraulics_reference	delta_t_c_nom/min/max
supply_pressure_mpa	PT01	facility.container_hydraulics_reference	supply_pressure_mpa_nom/min/max
return_pressure_mpa	PT02	facility.container_hydraulics_reference	return_pressure_mpa_nom/min/max
filter_before_mpa	PT03	facility.container_hydraulics_reference	filter_before_mpa_nom/max
filter_after_mpa	PT04	facility.container_hydraulics_reference	filter_after_mpa_nom/min
high_pressure_mpa	PT05	facility.container_hydraulics_reference	high_pressure_mpa_max
flow_rate_m3h	FT01	facility.container_hydraulics_reference	flow_rate_m3h_nom/min/max
conductivity_us	ET01	facility.container_hydraulics_reference	conductivity_us_cm_nom/max

Cooling Equipment (5 fields)

Container Monitor Field	Catalog Table	Catalog Column(s)
dry_cooler_freq_hz	facility.container_cooling_equipment	dry_cooler_freq_hz_nom/min/max
dry_cooler_on	facility.container_cooling_equipment	dry_cooler_count, dry_cooler_type
pump_1_freq_hz / pump_2_freq_hz	facility.container_cooling_equipment	pump_count, pump_type, pump_flow_rate_m3h
fan_status (4 fans)	facility.container_cooling_equipment	fan_count, fan_type

Power Monitoring (6 fields)

Container Monitor Field	Catalog Table	Catalog Column(s)
pmm1_kw (Racks 1-3)	facility.container_cooling_equipment	pmm_layout JSONB, total_power_capacity_kw
pmm2_kw (Racks 4-6)	facility.container_cooling_equipment	pmm_layout JSONB
pmm3_kw (Infrastructure)	facility.container_cooling_equipment	pmm_layout JSONB

Container Monitor Field	Catalog Table	Catalog Column(s)
asic_total_kw	facility.container_cooling_equipment	total_power_capacity_kw
infra_kw	facility.container_cooling_equipment	pmm_layout JSONB
pue	facility.container_cooling_equipment	pue_target/typical/worst_case

Environment (5 fields)

Container Monitor Field	Sensor	Catalog Table	Catalog Column(s)
inside_temp_c (x2)	TT31, TT33	facility.container_environment_reference	inside_temp_c_nom/max
dist_cabinet_temp_c	TT41	facility.container_environment_reference	dist_cabinet_temp_c_max
ctrl_cabinet_temp_c	TT43	facility.container_environment_reference	ctrl_cabinet_temp_c_max
outside_temp_c	External	facility.weather_reference	temp_f_avg (monthly baselines)
outside_humidity_pct	External	facility.weather_reference	humidity_pct_avg

Safety (5 fields)

Container Monitor Field	Catalog Table	Catalog Column(s)
tank_level_ok	facility.container_environment_reference	tank_level_sensor_present, safety notes
leakage_detected	facility.container_environment_reference	leakage_sensor_present, safety notes
smoke_detected	facility.container_environment_reference	smoke_sensor_present, safety notes
emergency_alarms []	ops.alert_rules	Alert threshold triggers, severity mapping
warning_alarms []	ops.alert_rules	Alert threshold triggers, severity mapping

12. knowledge.json Promotion Workflow

knowledge.json is Mining Guardian's working memory — updated every scan cycle with fleet summaries, discovered patterns, miner profiles, and LLM insights. The Intelligence Catalog is the validated, permanent version. Data moves from knowledge.json into the catalog through a defined promotion workflow.

Promotion Triggers

Patterns graduate from knowledge.json to the Intelligence Catalog through three channels:

Channel	Trigger	Process	Confidence
Bobby Manual Review	Bobby verifies a pattern	Bobby marks a knowledge.json pattern as confirmed. Next sync writes it to the catalog with verified_by_bobby = TRUE.	high
Weekly Claude Batch	Sunday 3am Tier 2 analysis	Claude reviews all accumulated patterns, cross-references against existing catalog entries, and promotes patterns that meet confidence thresholds. Writes to knowledge.llm_analysis_patterns with accuracy tracking.	medium
Auto-Promotion	Pattern accuracy exceeds 80% over 20+ predictions	knowledge.llm_analysis_patterns tracks total_predictions and correct_predictions. When accuracy_pct > 80 and total_predictions > 20, the pattern is flagged auto_action_eligible = TRUE. Low-accuracy patterns (< 40%) are deprioritized.	computed

Promotion Lifecycle

1. Observed — Pattern first appears in knowledge.json during a scan analysis or daily deep dive. No catalog entry yet.
2. Accumulated — Pattern is seen repeatedly across multiple scans/days. knowledge.json records occurrence count and affected miners.
3. Reviewed — Bobby or Claude evaluates the pattern. If valid, a row is inserted into the catalog with source citation (knowledge.sources, knowledge.citations) and confidence level.
4. Verified — After N successful predictions, the pattern's accuracy_pct in knowledge.llm_analysis_patterns is updated. High-accuracy patterns become auto-action eligible.
5. Permanent — Verified patterns with citations become permanent reference data. They survive knowledge.json resets, LLM retraining, and system migrations.

The key distinction: knowledge.json is ephemeral working memory that the LLM updates freely. The Intelligence Catalog is curated, citation-tracked, and permanent. The promotion workflow is the bridge between them.

13. Repair Data Ingestion

The repair schema (10 tables, 257 columns) is designed to ingest data from external repair shops like ACS (James) and Bobby's own bench work. This section documents the planned ingestion architecture.

The Flexible Ingestion Problem

Every repair shop provides data in a different format — spreadsheets, PDFs, emails, photos, handwritten notes. The catalog handles this with a two-layer approach:

Layer 1: Raw Capture

repair.repair_records has a raw_shop_data JSONB column that stores the exact data received from the shop in its original structure. No transformation, no data loss. Whatever format James sends — CSV columns, invoice line items, email text — it goes into raw_shop_data as-is. This column has no enforced schema; it is a flexible container.

Layer 2: Structured Extraction

The remaining 42 columns of repair.repair_records provide the structured, queryable version: miner_id, model, board_index, fault_description, parts_replaced (JSONB array), labor_hours, total_cost, outcome, warranty_status, etc. These columns are populated by an ETL process (manual initially, automated later) that reads raw_shop_data and maps fields to the structured schema.

Planned ETL Process

1. Receive — Bobby receives repair data from shop (email, spreadsheet, invoice, etc.).
2. Ingest — Raw data is inserted into repair.repair_records with raw_shop_data populated and a source tag.
3. Map — ETL script (or Bobby manually) extracts structured fields from raw_shop_data into the typed columns.
4. Enrich — Cross-reference with hardware tables: look up miner model, board serial batch, PSU batch. Link repair to known failure patterns.
5. Aggregate — repair.repair_statistics automatically updates with per-model, per-shop, per-failure-type success rates and costs.

Status: The schema is ready. The ETL scripts will be built once the first real repair data arrives from ACS and the format is known. The raw_shop_data JSONB approach ensures we can start ingesting immediately without waiting for a formal data agreement.

14. Data Protection

90 tables, 2,363 columns, and a 10-year retention horizon require a robust backup and recovery strategy. The Intelligence Catalog follows the 3-2-1 backup rule: three copies, two media types, one off-site.

Backup Strategy

Layer	Method	Frequency	Retention
Primary	Live database on ROBS-PC 2 TB SSD (Phase 1) / NAS RAID 5/6 (Phase 2)	Continuous	Live
Local Backup	pg_dump to D:\miner-intelligence\backups\ (Phase 1) / NAS share (Phase 2)	Daily	14 days
Off-site	Encrypted pg_dump upload to Backblaze B2	Nightly	30 days
Git Archive	Schema SQL files in GitHub repo (intelligence/database/)	On change	Permanent

Point-in-Time Recovery

PostgreSQL 16 supports WAL (Write-Ahead Logging) archiving for point-in-time recovery. Configuration:

- WAL archiving enabled in postgresql.conf: archive_mode = on, archive_command copies WAL segments to a separate directory.
- Base backups taken weekly with pg_basebackup for full filesystem snapshot.
- Recovery target: any point in time between the last base backup and the most recent WAL segment. If a bad import corrupts data at 3:47pm, restore to 3:46pm.
- Phase 2 (NAS): WAL segments archive to a separate NAS volume from the live database, providing media separation.

Disaster Recovery

Worst-case scenarios and recovery paths:

Scenario	Recovery	RPO	RTO
SSD failure on ROBS-PC	Restore latest pg_dump to new drive	< 24 hours	~1 hour
ROBS-PC total loss	Restore from Backblaze B2 to any Postgres host	< 24 hours	~2 hours
Accidental data corruption	Point-in-time recovery via WAL replay	Minutes	~30 min
Schema needs rebuild	Re-run 3 SQL files from GitHub repo	Schema only	~5 min

Data Export

The Intelligence Catalog can be exported in multiple formats for portability:

- `pg_dump` (SQL) — Full logical backup. Portable to any PostgreSQL 16+ instance. Default export format.
- `pg_dump` (custom) — Compressed binary format (-Fc). Faster restore with `pg_restore`, supports parallel restore.
- CSV export — Per-table CSV via COPY TO. For external analysis in Excel, Python, or other tools.
- JSON export — `schema_inventory.json` already provides machine-readable schema metadata. Table data can be exported as JSON via `psql` or custom scripts.

15. Security

Network Access

The Intelligence Catalog is accessible via two network paths. The correct path depends on where the client is:

Client Location	Connect Via	Address	When to Use
Facility LAN	Direct LAN	192.168.188.47:5432	Bobby's workstation, other LAN devices
VPS / Mac mini	Tailscale	100.110.87.1:5432	Mining Guardian production (over Tailscale mesh)
Remote (Bobby)	Tailscale	100.110.87.1:5432	Bobby's laptop when off-site

Database Roles

Role	Permissions	Used By
miner_intel (owner)	Full DDL + DML on all schemas. Creates tables, alters schema, manages roles.	Bobby (admin), migration scripts, schema deployments
guardian_reader	SELECT only on all schemas. Cannot INSERT, UPDATE, DELETE, or ALTER.	Mining Guardian production (context builder queries)
ingestion_writer	SELECT + INSERT + UPDATE on knowledge, market, repair schemas. No DDL.	Automated import scripts, LLM promotion workflow

Security Layers

- Authentication: Strong random password (32 characters), stored only in D:\miner-intelligence\.env on ROBS-PC. Never committed to git.
- Network isolation: Postgres binds to 192.168.188.47 only (LAN interface). No public internet exposure, no Cloudflare tunnel, no exposed ports.
- Tailscale ACLs: Only authorized Tailscale nodes (VPS, Mac mini, Bobby's devices) can reach ROBS-PC. Tailscale provides WireGuard encryption in transit.
- Backup encryption: Backup files are encrypted with a separate passphrase before upload to Backblaze B2.
- No shared credentials: Each role (owner, reader, writer) has its own password. Mining Guardian production uses guardian_reader with SELECT-only access.

Appendix A: SQL Query Examples

These are the common query patterns Mining Guardian will use when accessing the Intelligence Catalog. Each query shows how the context builder assembles reference data for AI decision-making.

Q1: Get baseline for a specific miner configuration

```
SELECT hashrate_th_baseline, power_w_baseline, efficiency_j_th_baseline,  
       hashrate_warning_pct, hashrate_critical_pct  
FROM ops.miner_baseline_reference  
WHERE model = 'S19JPro'  
       AND firmware = 'BixBiT'  
       AND cooling_type = 'immersion';
```

Used by: Outcome Feedback Loop, Pre-Failure Prediction

Q2: Check if a PSU batch is known for issues

```
SELECT batch_id, production_date_start, failure_rate_pct,  
       quality_rating, known_issues, recommended_action  
FROM hardware.psu_serial_batches  
WHERE manufacturer = 'Bitmain'  
       AND model = 'APW12'  
       AND serial_prefix = 'HJK23';
```

Used by: Pre-Failure Prediction, Intelligent Procurement

Q3: Look up failure patterns for declining hashrate

```
SELECT fp.name, fp.category, fp.severity, fp.root_cause,  
       fp.occurrence_rate_pct, fp.repair_success_rate_pct  
FROM ops.failure_patterns fp  
JOIN ops.symptom_pattern_map spm ON fp.id = spm.pattern_id  
JOIN ops.failure_symptoms fs ON spm.symptom_id = fs.id  
WHERE fs.name = 'gradual_hashrate_decline'  
ORDER BY spm.probability DESC;
```

Used by: Pre-Failure Prediction, Action Diversity

Q4: Get chip diagnostic reference values

```
SELECT chip_name, diode_mode_reading_v_nom, diode_mode_reading_v_min,  
       diode_mode_reading_v_max, domain_resistance_ohm_nom,  
       domain_resistance_ohm_min, domain_resistance_ohm_max  
FROM hardware.chips  
WHERE chip_name = 'BM1366';
```

Used by: Fingerprint Builder, Pre-Failure Prediction

Q5: Check for firmware known bugs

```
SELECT bug_id, severity, affected_versions, symptom,
       workaround, fixed_in_version
FROM firmware.firmware_bugs
WHERE firmware_family = 'BiXBiT'
      AND status = 'open'
      AND affected_models @> ARRAY['S19JPro'];
```

Used by: Firmware Regression Detection

Q6: Get repair procedures for a specific failure

```
SELECT rp.name, rp.difficulty_level, rp.estimated_time_minutes,
       rp.tools_required, rp.success_rate_pct, rp.estimated_cost_usd
FROM repair.repair_procedures rp
JOIN ops.failure_patterns fp ON rp.failure_pattern_id = fp.id
WHERE fp.name = 'psu_capacitor_degradation'
ORDER BY rp.success_rate_pct DESC;
```

Used by: Action Diversity, Repair-vs-Replace

Q7: Calculate depreciation for tax reporting

```
SELECT asset_type, method, useful_life_years,
       year_1_pct, year_2_pct, year_3_pct, year_4_pct, year_5_pct,
       section_179_eligible, bonus_depreciation_pct
FROM market.depreciation_schedules
WHERE asset_type = 'asic_miner'
      AND tax_jurisdiction = 'US_federal'
      AND effective_date <= CURRENT_DATE
ORDER BY effective_date DESC LIMIT 1;
```

Used by: Tax Compliance, Financial Reporting

Q8: Get container hydraulics reference for anomaly detection

```
SELECT supply_temp_c_nom, supply_temp_c_min, supply_temp_c_max,
       delta_t_c_nom, delta_t_c_min, delta_t_c_max,
       flow_rate_m3h_nom, conductivity_us_cm_max
FROM facility.container_hydraulics_reference
WHERE container_model = 'BiXBiT'
      AND cooling_type = 'immersion';
```

Used by: HVAC/Environment Correlation

Q9: Find high-accuracy LLM prediction patterns

```
SELECT pattern_category, pattern_description, accuracy_pct,
       total_predictions, avg_lead_time_hours,
```

```

    auto_action_eligible
FROM knowledge.llm_analysis_patterns
WHERE accuracy_pct > 80
    AND total_predictions > 20
ORDER BY accuracy_pct DESC;

```

Used by: Continuous Self-Improvement

Q10: Check if an unknown field has been seen before

```

SELECT raw_field_name, source_system, first_observed_at,
    sample_values, llm_suggested_category, status
FROM knowledge.unknown_fields
WHERE raw_field_name ILIKE '%newFieldName%'
    OR raw_field_name ILIKE '%new_field_name%';

```

Used by: Auto-Discovery Mechanism

Q11: Curtailment decision — revenue vs demand charge savings

```

SELECT dr.program_name, dr.capacity_payment_kw_month,
    dr.energy_payment_kwh, dr.estimated_annual_revenue_usd,
    er.demand_charge_kw_month
FROM facility.demand_response_programs dr
CROSS JOIN facility.electricity_rates er
WHERE dr.facility_id = (SELECT id FROM facility.facilities WHERE name = 'BiXBiT Fort Worth')
    AND er.facility_id = dr.facility_id
    AND er.effective_date <= CURRENT_DATE
ORDER BY er.effective_date DESC LIMIT 1;

```

Used by: Automated Curtailment Decisions

Q12: Repair-vs-replace total cost comparison

```

SELECT m.name AS model,
    ph.price_usd AS replacement_cost,
    rv.value_usd AS current_resale_value,
    ds.year_3_amount_usd AS book_value_year3
FROM hardware.miner_models m
LEFT JOIN market.pricing_history ph ON ph.model_id = m.id
    AND ph.condition = 'new' AND ph.recorded_at = (
    SELECT MAX(recorded_at) FROM market.pricing_history WHERE model_id = m.id)
LEFT JOIN market.resale_value_history rv ON rv.model_id = m.id
    AND rv.condition = 'used_good' AND rv.recorded_at = (
    SELECT MAX(recorded_at) FROM market.resale_value_history WHERE model_id = m.id)
LEFT JOIN market.depreciation_schedules ds ON ds.asset_type = 'asic_miner'
WHERE m.name = 'Antminer S19J Pro';

```

Used by: Repair-vs-Replace Decision Engine